```
RAG agent
# You may need to add your working directory to the Python path. To do so,
uncomment the following lines of code
# import sys
# sys.path.append("/Path/to/directory/agentic-framework") # Replace with
your directory path

import logging

from langchain_community.embeddings import OpenAIEmbeddings
from langchain_community.vectorstores import Chroma
from langchain_text_splitters import RecursiveCharacterTextSplitter

from baf import nlp
from baf.core.agent import Agent
from baf.core.session import Session
from baf.exceptions.logger import logger
from baf.nlp.llm.llm_huggingface_api import LLMHuggingFaceAPI
from baf.nlp.llm.llm_openai_api import LLMOpenAI
from baf.nlp.llm.llm_replicate_api import LLMReplicate
from baf.nlp.rag.rag import RAGMessage, RAG

# Configure the logging module (optional)
logger.setLevel(logging.INFO)

# Create the agent
agent = Agent('rag_agent')
# Load agent properties stored in a dedicated file
agent.load_properties('config.yaml')
# Define the platform your agent will use
websocket_platform = agent.use_websocket_platform(use_ui=True)

# Create Vector Store (RAG's DB)
vector_store: Chroma = Chroma(

embedding_function=OpenAIEmbeddings(openai_api_key=agent.get_property(nlp.O
PENAI_API_KEY)),
    persist_directory='vector_store'
)
# Create text splitter (RAG creates a vector for each chunk)
splitter = RecursiveCharacterTextSplitter(chunk_size=1000,
chunk_overlap=100)
# Create the LLM (for the answer generation)
gpt = LLMOpenAI(
    agent=agent,
    name='gpt-4o-mini',
    parameters={},
    num_previous_messages=10
)

# Other example LLM
```

```
# gemma = LLMHuggingFace(agent=agent, name='google/gemma-2b-it',
parameters={'max_new_tokens': 1}, num_previous_messages=10)
# llama = LLMHuggingFaceAPI(agent=agent,
name='meta-llama/Meta-Llama-3.1-8B-Instruct', parameters={},
num_previous_messages=10)
# mixtral = LLMReplicate(agent=agent,
name='mistralai/mixtral-8x7b-instruct-v0.1', parameters={},
num_previous_messages=10)

# Create the RAG
rag = RAG(
    agent=agent,
    vector_store=vector_store,
    splitter=splitter,
    llm_name='gpt-4o-mini',
    k=4,
    num_previous_messages=0
)
# Uncomment to fill the DB
# rag.load_pdfs('./pdfs')

# STATES

initial_state = agent.new_state('initial_state', initial=True)
rag_state = agent.new_state('rag_state')


# STATES BODIES' DEFINITION + TRANSITIONS

def initial_body(session: Session):
    session.reply('Hi!')


initial_state.set_body(initial_body)
# TODO : fix no_intent_matched
initial_state.when_no_intent_matched().go_to(rag_state)


def rag_body(session: Session):
    rag_message: RAGMessage = session.run_rag(session.event.message)
    websocket_platform.reply_rag(session, rag_message)


rag_state.set_body(rag_body)
rag_state.go_to(initial_state)


# RUN APPLICATION

if __name__ == '__main__':
    agent.run()
```